CSCI 2800 — Computer Architecture and Operating Systems (CAOS)
Homework 3 (document version 1.0)

- This homework is due in Submitty by 10:00PM Thursday, November 19, 2026
- You can use at most **two late days** on this assignment
- This homework is to be done individually, so **please do not share your code with others**
- Please do **not** use AI to assist you on this homework
- Place all of your code in `hw3.c` for submission; no other files may be included
- You **must** use C for this assignment, and all submitted code **must** successfully compile via `gcc` with no warning messages when the `-Wall` (i.e., warn all) compiler option is used; we will also use `-Werror`, which will treat all warnings as critical errors
- All submitted code **must** successfully compile and run on Submitty, which currently uses Ubuntu v22.04.5 LTS and `gcc` version 11.4.0 (`Ubuntu 11.4.0-1ubuntu1~22.04.3`)
- This assignment will be entirely auto-graded by Submitty; you will have **eight** penalty-free submissions, after which points will slowly be deducted, e.g., `-1` on submission #9, etc.
- You will have at least **three** days before the due date to start submitting your code to Submitty; if auto-grading is not available three days before the due date, the due date will be 10:00PM three days after auto-grading becomes available

Hints and reminders

Do **not** rely on program output to confirm whether your code is correct. Consistently allocate **exactly** the number of bytes you need regardless of whether you use static or dynamic memory allocation. Further, deallocate dynamically allocated memory via `free()` at the earliest possible point in your code.

Make use of `valgrind` (or `drmemory` or other dynamic memory checkers) or the "valgrind unlimited" gradeable in Submitty to check for errors with dynamic memory allocation and dynamic memory usage. As another helpful hint, close open file descriptors as soon as you are done using them.

Always read (and re-read!) the `man` pages for library functions (section 3), system calls (section 2), etc., especially if you do not recognize a function you are required to use.

To understand how `man` pages are organized, check out the `man` page for `man` itself (via `man man`).

Homework specifications

In this third homework, you will use C to implement a MIPS mini-assembler. Given an input file of valid MIPS assembly instructions (per the specifications that follow), you will produce the associated 32-bit words that make up the machine code—this will be text to `stdout` consisting of the characters `'0'` and `'1'`.

No square brackets allowed

To emphasize the use of pointers and pointer arithmetic, **you are not allowed to use square brackets** anywhere in your code! If a `'['` or `']'` character is detected in your code, **including within comments,** Submitty will remove that entire line from your code before running `gcc`.

Input specifications

Your program should expect a single command-line argument that will be the name of the assembly file to read, e.g., `"mips2.s"`. This file will be standard ASCII text with one MIPS instruction per non-blank line. Blank lines in the input file should be ignored.

A non-blank line will have the following items in this order:

- An optional label. Labels will be ALL-CAPS (no numerals or spaces) followed by a colon and zero or more whitespace characters; use `isspace()` for this. You are guaranteed the same label will not appear twice in one input file; colon characters are not used for any other purpose.
- A MIPS instruction (or pseudo-instruction—see below). These will be all lowercase letters from the list of instructions given later in this document, which is a subset of the full MIPS instruction set.
- A single space character.
- The operands for the specified instruction. Most operands are separated by commas, but some instructions (notably `lw` and `sw`) will also use other symbols. There will be no whitespace characters separating operands.
  - Register operands will always start with a `'$'` character and will use the names given on the MIPS reference card, e.g., `$t0`, `$a0`, etc.
  - Immediate operands will be numerals, possibly with a negative sign in front. You are guaranteed that the value will not exceed the bounds for minimum and maximum values for that particular instruction.
  - LABEL operands (as in a branch instruction) are guaranteed to exist somewhere in the input file.
- Zero or more whitespace characters, followed by an optional comment. Comments always start with a `'#'` character, and may be followed by anything other than a newline (`'\n'`). They should simply be ignored.
- A newline (`'\n'`) character ending the line.

Note that there will be no lines containing just a label; similarly, no lines will consist of just a comment—every non-blank line will contain an instruction. And you are guaranteed that each line will be valid MIPS code, and that every input file will contain at least one instruction. You are not guaranteed that the input file will be a useful program—it could have infinite loops, etc., but that is not relevant for this task. All you need to do is parse the MIPS code and turn it into its binary machine code.

As per usual, you are required to use `open()`, `read()`, and `close()` to access the input file. Consider reusing code from your previous homework assignments.

Output specifications

Your program should write all of its results to standard output (`stdout`). The output should be a series of lines consisting of precisely 32 characters, all of which are either `'0'` (ASCII `0x30`) or `'1'` (ASCII `0x31`). The lines should correspond to the machine code corresponding to the MIPS code in the input file. When this is complete, the program should write one additional line containing only the word `"end"` and then terminate.

Instruction set (simple)

A program that correctly assembles all of the following instructions (but nothing in later sections) is worth a grade of 60%.

```
add,addi,and,andi,jr,lw,nor,or,ori,sb,sh,sw,sub
```

Note that in all cases (including instructions listed further down), any unused operand should have all of its bits set to zero. For example, all instructions above do not use the `shmat` operand; therefore, the unused `shmat` field should be set to all `'0'` characters. Similarly, the `sll` instruction (see below) is an R-format instruction that uses the `rd`, `rt`, and `shmat` operands; therefore, the unused `rs` field should be set to all `'0'` characters.

Instruction set (adding shifts, branches, and jumps)

A program that correctly assembles all of the following instructions plus the ones above, but no pseudo-instructions, is worth a grade of 85%.

```
beq,bne,j,jal,sll,srl
```

These instructions add three additional types of instructions. Shift instructions should be fairly clear—they just use the `shamt` field of the R format, but otherwise work similarly to dealing with other immediate values. (The shift amount is unsigned, thus 0 to 31.)

The branch instructions have a label as their third operand. Your program must locate this label and come up with the appropriate immediate value to indicate how far away that label is in the code. Remember that we count from the line after the branch instruction, so if the label is five instructions before the branch, the correct immediate value will be `-6`.

Jump instructions also have a label, but that label is an absolute memory address, not a relative location. We will assume for this homework that the first instruction is located at memory address `0x00400000` (just like in `spim`). You will then need to count instructions to find the correct memory address of the label. Remember that all instruction addresses are multiples of 4 (i.e., they end in `00`); therefore, `00` is not written into the binary for the jump instructions. Look carefully at Note (5) on the MIPS reference card where it shows you what a `JumpAddr` is for more details.

Pseudo-instructions

To be eligible for the maximum grade of 100%, a program must correctly assemble all of the instructions mentioned above plus these pseudo-instructions.

```
bge,bgt,ble,blt,li,move,subi
```

The four conditional branch pseudo-instructions must be translated into two separate instructions: an `slt` followed by either `beq` or `bne`. It is up to you to place the operands correctly and to select the correct one, i.e., `beq` or `bne`. You **must use** the `$at` register as your temporary register in both of these instructions.

The `li` pseudo-instruction behaves differently depending on the size of the immediate value. You can assume that the immediate value fits in 16 bits, i.e., is a value ranging from -32768 to 32767.

The `move` pseudo-instruction should use an `add` instruction to perform its task, and the `subi` pseudo-instruction should perform an `addi` instruction with a value of the opposite sign.

In all cases, if you need to use `$zero` as part of your pseudo-instruction, it should go in the `rt` field.

Pseudo-instruction examples

Below are a number of pseudo-instruction substitutions.

```
# subi $t1,$t2,13     # $t1 = $t2 - 13
addi $t1,$t2,-13    # $t1 = $t2 + -13

# subi $t1,$t2,-73
addi $t1,$t2,73

# subi $t1,$t2,0
addi $t1,$t2,0


# move $s1,$s2        # $s1 = $s2
add $s1,$s2,$zero   # $s1 = $s2 + 0


# li $s1,88           # immediate value is within 16 bits
ori $s1,$zero,88    # $s1 = 0 | 88

# li $s1,-8           # immediate value is within 16 bits
addi $s1,$zero,-8   # $s1 = 0 + -8


# bge $s1,$s2,LABEL   # branch if $s1 >= $s2
slt $at,$s1,$s2     # $at is 1 if $s1 < $s2; 0 otherwise ($s1 >= $s2)
beq $at,$zero,LABEL

# ble $s1,$s2,LABEL   # branch if $s1 <= $s2
slt $at,$s2,$s1     # $at is 1 if $s2 < $s1; 0 otherwise ($s1 <= $s2)
beq $at,$zero,LABEL

# blt $s1,$s2,LABEL   # branch if $s1 < $s2
slt $at,$s1,$s2     # $at is 1 (not equal to 0) if $s1 < $s2
bne $at,$zero,LABEL

# bgt $s1,$s2,LABEL   # branch if $s1 > $s2
slt $at,$s2,$s1     # $at is 1 (not equal to 0) if $s2 < $s1
bne $at,$zero,LABEL
```

Note that if a pseudo-instruction begins with a label, the label is assigned to the first instruction in the set of instructions that you generate.

Example program execution

Below is an example showing the output format you must follow.

```
bash$ cat hw3-input01.s
addi $t1,$zero,-6
add $t2,$t1,$t0
bash$ ./hw3.out hw3-input01.s
00100000000010011111111111111010
00000001001010000101000000100000
end
bash$
```

Error handling

If an improper command-line argument is given, report an error message to `stderr` and exit the program with `EXIT_FAILURE`. In general, if an error is encountered, display a meaningful error message on `stderr` by using either `perror()` or `fprintf()`, then exit the program with `EXIT_FAILURE`. Only use `perror()` if the given library or system call sets the global `errno` variable.

Error messages must be one line only and use the following format:

```
ERROR: <error-text-here>
```

Submitty may check for the existence of the line above by verifying that the line starts with the `"ERROR:"` string.

Submission instructions

To submit your assignment and perform **final** testing of your code, we will use the auto-grading features of Submitty. Your code will not be manually reviewed.

To help ensure that your program executes properly, consider making use of the `DEBUG_MODE` preprocessor technique shown below. This helps to avoid accidentally displaying extraneous output in Submitty. Here is an example:

```
#ifdef DEBUG_MODE
    printf( "the value of x is %d and y is %f\n", x, y );
    printf( "here123\n" );
    printf( "why is my program crashing here?!\n" );
#endif
```

To compile this code in "debug" mode, use the `-D` flag as follows:

```
bash$ gcc -Wall -Werror -g -D DEBUG_MODE hw3.c
```

Submitty will use the following to compile your code:

```
bash$ gcc -Wall -Werror hw3.c
```

Also, Submitty uses buffered output when writing data; therefore, add the line below as the first statement in your `main()` function to have all output sent to `stdout` be non-buffered. If your program hangs or crashes, this will ensure that you will see output to that point.

```
setvbuf( stdout, NULL, _IONBF, 0 );
```